

```

28  def __init__(
29      self,
30      transitions,
31      start_state="q0",
32      accept_state="qa",
33      reject_state="qr",
34      blank_symbol="",
35  ):
36      # setting turing machine parameters
37      self.transitions = transitions
38      self.start_state = start_state
39      self.accept_state = accept_state
40      self.reject_state = reject_state
41      self.blank_symbol = blank_symbol
42
43  def run(self, input_):
44      """Execute the Turing machine for a particular input.
45
46      :param input_: the input that is written on the tape. It can be a list
47      of strings, or just a string, in which case each letter is treated as a symbol.
48
49      This method MUST be a Python generator. It should yield a (action, configuration) tuple
50      at each step of the computation.
51
52      The action is either 'Accept', 'Reject' or None.
53
54      Configuration is a dictionary with the following keys:
55      - 'state': the current state,
56      - 'left_hand_side': list of symbols on the left hand side of the current position (closest first),
57      - 'symbol': the current symbol under the head,
58      - 'right_hand_side': list of symbols on the right hand side of the current position.
59
60      """
61      # check if the input is str, if it is convert it to list
62      if isinstance(input_, str):
63          tape = list(input_)
64      else:

```

```

61      # check if the input is str, if it is convert it to list
62      if isinstance(input_, str):
63          tape = list(input_)
64      else:
65          tape = input_
66
67      # calculating left and right tape of the current symbol using two lists (left list is invert)
68      left = []
69      if tape:
70          symbol = tape[0]
71          right = tape[1:]
72      else:
73          symbol = self.blank_symbol
74          right = []
75
76      current_state = self.start_state
77      # returning the initial state
78      yield (
79          None,
80          {
81              "state": current_state,
82              "left_hand_side": list(left),
83              "symbol": symbol,
84              "right_hand_side": list(right),
85          },
86      )
87
88      while True:
89
90          # input accepted
91          if current_state == self.accept_state:
92              yield (
93                  "Accept",
94                  {

```

```

90         # input accepted
91         if current_state == self.accept_state:
92             yield (
93                 "Accept",
94                 {
95                     "state": current_state,
96                     "left_hand_side": list(left),
97                     "symbol": symbol,
98                     "right_hand_side": list(right),
99                 },
100             )
101         return
102
103         # input rejected
104         if current_state == self.reject_state:
105             yield (
106                 "Reject",
107                 {
108                     "state": current_state,
109                     "left_hand_side": list(left),
110                     "symbol": symbol,
111                     "right_hand_side": list(right),
112                 },
113             )
114         return
115
116         # go in transition
117
118         # current transition can be made
119         key = (current_state, symbol)
120         if key not in self.transitions:
121             # we are not in accept state and halt which means reject
122             yield (
123                 "Reject",
124                 {

```

```

120         if key not in self.transitions:
121             # we are not in accept state and halt which means reject
122             yield (
123                 "Reject",
124                 {
125                     "state": current_state,
126                     "left_hand_side": list(left),
127                     "symbol": symbol,
128                     "right_hand_side": list(right),
129                 },
130             )
131         return
132
133         next_state, write_symbol, direction = self.transitions[key]
134
135         # write new symbol to input
136         symbol = write_symbol
137
138         # move head
139         if direction == "R":
140             # add the symbol to left handside tape and go right
141             left.insert(0, symbol)
142             if right:
143                 symbol = right[0]
144                 right.pop(0)
145             else:
146                 symbol = self.blank_symbol
147
148         elif direction == "L":
149             # add the symbol to right handside tape and go left
150             right.insert(0, symbol)
151             if left:
152                 symbol = left[0]
153                 left.pop(0)
154             else:
155                 symbol = self.blank_symbol

```

```

156
157         else:
158             # neither of R or L so it is bad input
159             yield (
160                 "Bad Input",
161                 {
162                     "state": current_state,
163                     "left_hand_side": list(left),
164                     "symbol": symbol,
165                     "right_hand_side": list(right),
166                 },
167             )
168             return
169
170     # move to next state
171     current_state = next_state
172
173     # yield the result
174     yield (
175         None,
176         {
177             "state": current_state,
178             "left_hand_side": list(left),
179             "symbol": symbol,
180             "right_hand_side": list(right),
181         },
182     )
183

```

در این بخش، سازنده‌ی کلاس و متدهای اصلی اجرای ماشین تورینگ پیاده‌سازی شده است.

در ابتدای این بلاک، متد سازنده‌ی کلاس قرار دارد. در تمپلیت، امضای این متد و پارامترهای اصلی آن مشخص شده بود. این پارامترها شامل `start_state`، `accept_state`، `reject_state`، `transitions` و `blank_symbol` هستند. در نسخه‌ی کامل‌شده، این مقادیر به عنوان ویژگی‌های شیء ذخیره شده‌اند تا در بقیه‌ی اجرای ماشین قابل استفاده باشند. این بخش اگرچه از نظر ظاهری ساده است، اما نقش مهمی در عمومی‌بودن کلاس دارد. به کمک این طراحی، کلاس فقط مخصوص یک ماشین خاص نیست؛ بلکه می‌توان با تغییر دیکشنری `transitions`، ماشین‌های متفاوتی را با همین شبیه‌ساز اجرا کرد. برای مثال ماشین تشخیص یک رشته، ماشین جمع یکانی، ماشین ضرب یکانی یا Busy Beaver همگی فقط با تغییر قوانین انتقال قابل تعریف هستند.

پارامتر `transitions` مهم‌ترین ورودی سازنده است، زیرا رفتار واقعی ماشین تورینگ را مشخص می‌کند. این پارامتر یک دیکشنری از قوانین انتقال است که در آن هر کلید از دو بخش حالت فعلی و نماد خوانده‌شده تشکیل می‌شود، و مقدار متناظر آن شامل حالت بعدی، نماد جدید برای نوشتن روی نوار و جهت حرکت هد است. در نسخه‌ی تکمیل‌شده، این دیکشنری در `self.transitions` ذخیره شده تا در زمان اجرای متد `run` بتوان در هر گام قانون مناسب را از آن استخراج کرد. همچنین حالت شروع، حالت پذیرش، حالت رد و نماد خالی نیز ذخیره شده‌اند تا ماشین بتواند بدون وابستگی به مقادیر ثابت، برای سناریوهای مختلف استفاده شود.

بعد از سازنده، متد `run` پیاده‌سازی شده است. این متد هسته‌ی اصلی شبیه‌ساز محسوب می‌شود، چون اجرای واقعی ماشین تورینگ داخل آن انجام می‌گیرد. طبق خواسته‌ی پروژه، این متد باید به صورت `generator` نوشته می‌شد؛ یعنی به جای اینکه فقط نتیجه‌ی نهایی محاسبه را برگرداند، در هر مرحله وضعیت ماشین را تولید کند. در نسخه‌ی تکمیل‌شده، این کار با استفاده از `yield` انجام شده است. این موضوع یکی از قسمت‌های مهم پیاده‌سازی است، چون باعث می‌شود بتوان اجرای ماشین را گام‌به‌گام مشاهده کرد و در مصرف حافظه صرفه جویی می‌شود.

در ابتدای متد `run`، ورودی ماشین آماده‌سازی می‌شود. اگر ورودی از نوع رشته باشد، به لیستی از کاراکترها تبدیل می‌شود. دلیل این کار این است که در ماشین تورینگ، هر خانه‌ی نوار شامل یک نماد جداگانه است و شبیه‌ساز باید بتواند خانه‌به‌خانه روی ورودی حرکت کند. اگر ورودی از قبل به صورت لیست داده شده باشد، همان مقدار به عنوان نوار استفاده می‌شود. این بخش باعث می‌شود پیاده‌سازی انعطاف‌پذیرتر باشد و هم ورودی‌های رشته‌ای معمولی و هم ورودی‌های لیستی را پشتیبانی کند.

یکی از بخش‌هایی که در نسخه‌ی تکمیل‌شده اهمیت دارد، نحوه‌ی مدل‌سازی نوار ماشین است. به جای استفاده از یک لیست واحد همراه با اندیس `head`، نوار به سه قسمت تقسیم شده است: لیست سمت چپ `head`، نماد فعلی `cursor` زیر `head`، و لیست سمت راست `tail`. در ابتدا، اگر ورودی خالی نباشد، اولین نماد ورودی به عنوان نماد زیر `head` انتخاب می‌شود و بقیه‌ی نمادها در سمت راست قرار می‌گیرند. اگر ورودی خالی باشد، نماد فعلی برابر با `blank_symbol` قرار داده می‌شود. این طراحی با فرمت پیکربندی خواسته‌شده در داکيومنت پروژه هماهنگ است، چون خروجی هر گام باید شامل `left_hand_side`، `symbol` و `right_hand_side` باشد.

نکته‌ی مهم در این پیاده‌سازی این است که بخش چپ و راست نوار با دو لیست جداگانه نگهداری می‌شوند. این کار مدیریت حرکت `head` را ساده‌تر می‌کند. وقتی `head` به راست حرکت می‌کند، نماد فعلی وارد بخش چپ می‌شود و نماد بعدی از بخش راست خوانده می‌شود. وقتی `head` به چپ حرکت می‌کند، نماد فعلی وارد بخش راست می‌شود و نماد قبلی از بخش چپ خوانده می‌شود. در صورتی که در سمت مورد نظر نمادی وجود نداشته باشد، نماد خالی جایگزین می‌شود. این قسمت از پیاده‌سازی، علاوه بر اجرای ماشین‌های عادی، برای بخش سوم پروژه نیز اهمیت دارد؛ چون در `Busy Beaver` نیاز است نوار بتواند در دو جهت گسترش پیدا کند و ماشین هنگام حرکت به سمت چپ دچار خطا نشود.

پس از آماده‌سازی نوار، حالت فعلی ماشین برابر با حالت شروع قرار داده می‌شود و اولین پیکربندی ماشین تولید می‌شود. در این پیکربندی، مقدار `action` برابر `None` است، چون ماشین هنوز به حالت پذیرش یا رد نرسیده است. این خروجی اولیه برای مشاهده‌ی وضعیت شروع ماشین لازم است و باعث می‌شود اجرای `generator` از همان لحظه‌ی قبل از انجام اولین `transition` قابل بررسی باشد. این موضوع با خواسته‌ی پروژه که تولید گام‌به‌گام پیکربندی ماشین را لازم دانسته بود، کاملاً هماهنگ است.

در ادامه، حلقه‌ی اصلی اجرای ماشین آغاز می‌شود. در هر تکرار حلقه، ابتدا بررسی می‌شود که آیا ماشین به حالت پذیرش رسیده است یا نه. اگر حالت فعلی برابر با `accept_state` باشد، مقدار `Accept` همراه با پیکربندی فعلی تولید می‌شود و اجرای متد پایان می‌یابد. به همین شکل، اگر حالت فعلی برابر با `reject_state` باشد، مقدار `Reject` تولید شده و ماشین متوقف می‌شود. این بخش از کد نقش تعیین‌کننده‌ای در تشخیص پایان محاسبه دارد و مشخص می‌کند که ورودی توسط ماشین پذیرفته شده یا رد شده است.

اگر ماشین هنوز در حالت پذیرش یا رد نباشد، باید قانون انتقال مناسب پیدا شود. برای این کار، زوج `(current_state, symbol)` ساخته می‌شود؛ یعنی حالت فعلی ماشین و نمادی که زیر `head` قرار دارد. سپس این زوج در دیکشنری `transitions` جست‌وجو می‌شود. این بخش دقیقاً همان منطق رسمی ماشین تورینگ را پیاده‌سازی می‌کند؛ زیرا در هر گام تصمیم ماشین فقط به حالت فعلی و نماد

خوانده شده بستگی دارد. اگر برای این وضعیت قانونی وجود نداشته باشد، ماشین نمی تواند ادامه دهد. در پیاده سازی انجام شده، این حالت به عنوان رد ورودی در نظر گرفته شده و خروجی Reject تولید می شود. این کار باعث می شود شبیه ساز در وضعیت های تعریف نشده به شکل کنترل شده متوقف شود و اجرای برنامه با خطاهای ناخواسته مواجه نشود.

در صورتی که قانون انتقال وجود داشته باشد، سه مقدار `next_state`، `write_symbol` و `direction` از آن استخراج می شود. ابتدا نماد فعلی نوار با `write_symbol` جایگزین می شود. این بخش عملیات نوشتن روی نوار را شبیه سازی می کند. سپس بر اساس مقدار `direction`، هد به سمت چپ یا راست حرکت می کند. اگر جهت حرکت R باشد، نماد نوشته شده به سمت چپ هد منتقل می شود و هد روی نماد بعدی در سمت راست قرار می گیرد. اگر سمت راست خالی باشد، نماد خالی به عنوان خانه ی جدید نوار در نظر گرفته می شود. بنابراین نوار در سمت راست به صورت پویا گسترش پیدا می کند.

در حالت حرکت به چپ نیز فرآیند مشابهی انجام شده است. نماد فعلی به ابتدای سمت راست منتقل می شود و اگر در سمت چپ نمادی وجود داشته باشد، هد روی آن قرار می گیرد. اگر سمت چپ خالی باشد، نماد خالی جایگزین می شود. این بخش مهم است زیرا فقط یک حرکت ساده ی هد نیست، بلکه منطق گسترش نوار را هم پیاده سازی می کند. با این روش، ماشین هنگام حرکت به خانه ای که قبلاً در نوار وجود نداشته است، همچنان می تواند به کار خود ادامه دهد و آن خانه را به عنوان یک خانه ی خالی در نظر بگیرد.

همچنین برای حالتی که جهت حرکت نه R باشد و نه L، یک خروجی با عنوان Bad Input در نظر گرفته شده است. این بخش برای کنترل خطا در تعریف قوانین انتقال استفاده می شود. چون طبق تعریف پروژه، جهت حرکت باید یکی از دو مقدار چپ یا راست باشد، هر مقدار دیگر نشان دهنده ی تعریف نامعتبر transition است. بنابراین این قسمت باعث می شود خطاهای احتمالی در قوانین انتقال بهتر قابل تشخیص باشند.

در انتهای هر گام، حالت فعلی ماشین به `next_state` تغییر داده می شود و سپس پیکربندی جدید ماشین با `yield` برگردانده می شود. این پیکربندی شامل حالت جدید، بخش چپ نوار، نماد زیر هد و بخش راست نوار است. در این مرحله مقدار `action` برابر `None` باقی می ماند، چون ماشین هنوز در حال اجراست و به حالت توقف نرسیده است. به این ترتیب، بعد از هر transition وضعیت جدید ماشین تولید می شود و کاربر می تواند روند محاسبه را مرحله به مرحله دنبال کند.

چرا استفاده از generator مناسب است؟

چون اجرای ماشین تورینگ ذاتاً مرحله به مرحله انجام می شود. در هر گام، ماشین یک نماد را از نوار می خواند، بر اساس حالت فعلی و قانون انتقال تصمیم می گیرد، نمادی را می نویسد، هد را جابه جا می کند و وارد حالت بعدی می شود. ژنراتور دقیقاً با همین مدل سازگار است، چون می تواند بعد از هر گام، وضعیت فعلی ماشین را با `yield` برگرداند و سپس اجرای محاسبه را از همان نقطه ادامه دهد.

مزیت مهم دیگر ژنراتورها این است که لازم نیست همه‌ی مراحل اجرای ماشین از ابتدا تا انتها در حافظه ذخیره شوند. بعضی ماشین‌های تورینگ، مخصوصاً ماشین‌های مربوط به **Busy Beaver**، ممکن است تعداد زیادی گام اجرا شوند یا حتی متوقف نشوند. اگر بخواهیم تمام مراحل را یک‌جا محاسبه و ذخیره کنیم، هم حافظه‌ی زیادی مصرف می‌شود و هم کنترل اجرای برنامه سخت‌تر می‌شود. اما با ژنراتور، هر بار فقط یک مرحله تولید می‌شود و کاربر یا متدهای دیگر می‌توانند تصمیم بگیرند چند گام از ماشین اجرا شود.

همچنین ژنراتورها برای **debug** کردن بسیار مناسب‌اند. چون شبیه‌ساز می‌تواند بعد از هر **transition**، پیکربندی ماشین را شامل حالت فعلی، نماد زیر هد، سمت چپ نوار و سمت راست نوار برگرداند. به این ترتیب می‌توان روند اجرای ماشین را مرحله‌به‌مرحله مشاهده کرد و فهمید که در هر انتقال چه تغییری در نوار و حالت ماشین ایجاد شده است.

```

184     def accepts(self, input_, step_limit=100):
185         """Check whether the Turing machine accepts a string.
186
187         :param input_: the input string or list.
188         :param step_limit: the maximum number of steps to simulate before stopping.
189         :return: True if the machine halts in accept_state, False if it rejects,
190                 or None if the step limit is reached without halting.
191         """
192         # creating the turing generator
193         turing_gen = self.run(input_)
194
195         # going through the step limits until machine halt
196         for _ in range(step_limit):
197             try:
198                 action, conf = next(turing_gen)
199                 if action == "Accept":
200                     return True
201                 elif action == "Reject":
202                     return False
203             except StopIteration:
204                 break
205         # logging that the machine did not halt
206         logging.warning(
207             "the step limit has reached and input not accepted nor rejected"
208         )
209         return None
210
211     def rejects(self, input_, **kwargs):
212         """Check whether the Turing machine rejects a string.
213
214         :param input_: the input string or list.
215         :return: True if the machine rejects the string, False if it accepts.
216         """
217         # just returning the not of the accept result
218         result = self.accepts(input_=input_)
219         if result is None:
220             return None

```

```

224     def debug(self, input_, step_limit=100, colored=False):
225         """Print the execution configuration of the machine per transition for debugging.
226
227         :param input_: the input string or list.
228         :param step_limit: the maximum number of steps to output.
229         :param colored: True to output colored boundaries in terminal.
230         """
231         halted = False
232         for step, (action, config) in enumerate(islice(self.run(input_), step_limit)):
233
234             # getting config properties
235             state = config["state"]
236             left = config["left_hand_side"]
237             right = config["right_hand_side"]
238             symbol = config["symbol"]
239
240             # make the left and right string (left should be reversed)
241             left = "".join(reversed(left))
242             right = "".join(right)
243             # construct the tape
244             if colored:
245                 # adding color to the current symbol
246                 tape = f"{left}\033[91m{symbol}\033[0m{right}"
247             else:
248                 tape = f"{left}{symbol}{right}"
249
250             if action is None:
251                 print(f"step {step}, state: {state}, symbol: {symbol}")
252                 print(f"\ttape: {tape}")
253                 print()
254             else:
255                 halted = True
256                 return
257         # logging that machine did not halt
258         if not halted:
259             logging.warning("Debug stopped after reaching the step limit.")

```

در این قسمت، متدهای مربوط به بررسی نتیجه‌ی اجرای ماشین و نمایش روند اجرای آن پیاده‌سازی شده‌اند. این بخش از کد روی خروجی `run` کار می‌کند؛ یعنی همان `generator` که اجرای ماشین تورینگ را مرحله‌به‌مرحله تولید می‌کند. هدف این قسمت این است که بتوانیم از روی اجرای ماشین تشخیص دهیم ورودی پذیرفته شده، رد شده، یا ماشین در تعداد گام مشخص شده هنوز متوقف نشده است. علاوه بر این، امکان نمایش وضعیت ماشین در هر مرحله نیز فراهم شده است تا بتوان اجرای ماشین را بهتر بررسی و اشکال‌زدایی کرد.

در متد `accepts` ابتدا با استفاده از `self.run(input_)` یک `generator` از اجرای ماشین ساخته می‌شود. این `generator` در هر مرحله یک مقدار `action` و یک `configuration` تولید می‌کند. مقدار `action` مشخص می‌کند که ماشین در آن مرحله به حالت پذیرش رسیده، رد شده، یا هنوز در حال اجراست. در نسخه‌ی تکمیل‌شده، این `generator` داخل یک حلقه اجرا می‌شود و تعداد دفعات اجرای آن با `step_limit` محدود شده است. دلیل استفاده از `step_limit` این است که بعضی ماشین‌های تورینگ ممکن است برای مدت زیادی اجرا شوند یا اصلاً متوقف نشوند؛ بنابراین اجرای ماشین نباید بدون محدودیت ادامه پیدا کند.

در هر تکرار حلقه، با دستور `next` یک مرحله از اجرای ماشین گرفته می‌شود. اگر مقدار `action` برابر `"Accept"` باشد، یعنی ماشین به حالت پذیرش رسیده است و متد مقدار `True` برمی‌گرداند. اگر مقدار `action` برابر `"Reject"` باشد، یعنی ماشین ورودی را رد کرده است و متد مقدار `False` برمی‌گرداند. به این ترتیب، متد `accepts` خروجی مرحله‌به‌مرحله‌ی ماشین را به یک نتیجه‌ی ساده و قابل استفاده تبدیل می‌کند.

در این پیاده‌سازی، حالت پایان `generator` نیز با `StopIteration` کنترل شده است. اگر `generator` به هر دلیل پایان پیدا کند، برنامه دچار خطا نمی‌شود و حلقه متوقف می‌شود. اگر ماشین تا رسیدن به حد `step_limit` نه قبول کند و نه رد شود، یک هشدار با استفاده از `logging.warning` ثبت می‌شود و مقدار `None` برگردانده می‌شود. این مقدار نشان می‌دهد که در تعداد گام‌های مشخص شده، نتیجه‌ی قطعی برای ورودی به دست نیامده است.

متد `rejects` بر اساس نتیجه‌ی متد `accepts` پیاده‌سازی شده است. در این بخش، ابتدا `accepts` برای ورودی اجرا می‌شود و نتیجه‌ی آن در متغیر `result` قرار می‌گیرد. اگر نتیجه `None` باشد، یعنی ماشین در محدوده‌ی گام مشخص شده نه پذیرفته و نه رد شده است؛ بنابراین این متد هم مقدار `None` برمی‌گرداند. در غیر این صورت، خروجی متد برابر نقیض نتیجه‌ی `accepts` است. یعنی اگر `accepts` مقدار `True` بدهد، `rejects` مقدار `False` برمی‌گرداند و اگر `accepts` مقدار `False` بدهد، `rejects` مقدار `True` برمی‌گرداند. این کار باعث می‌شود منطق بررسی پذیرش و رد جداگانه تکرار نشود و هر دو متد بر پایه‌ی یک روند اجرایی یکسان کار کنند.

در متد `debug` اجرای ماشین برای نمایش مرحله‌به‌مرحله آماده شده است. در نسخه‌ی تکمیل‌شده، ابتدا مقدار `halted` برابر `False` قرار داده می‌شود تا مشخص شود ماشین در طول اجرای محدودشده متوقف شده یا نه. سپس با استفاده از `islice` فقط تعداد مشخصی از مراحل `generator` گرفته می‌شود. این محدودیت با `step_limit` کنترل می‌شود تا اگر ماشین توقف نداشت، خروجی بی‌نهایت تولید نشود.

در هر مرحله از اجرای `debug`، اطلاعات پیکربندی ماشین از دیکشنری `config` استخراج می‌شود. این اطلاعات شامل حالت فعلی ماشین، بخش چپ نوار، بخش راست نوار و نماد زیر هد است. سپس بخش چپ و راست نوار به رشته تبدیل می‌شوند تا بتوان آن‌ها را در خروجی چاپ کرد. چون بخش چپ نوار در ساختار داخلی برنامه به صورت معکوس نگهداری می‌شود، قبل از چاپ با `reversed` به حالت درست نمایشی برگردانده می‌شود. این کار باعث می‌شود نوار در خروجی به همان ترتیبی دیده شود که کاربر انتظار دارد.

بعد از آماده سازی سمت چپ و راست نوار، رشته ی کامل نوار ساخته می شود. اگر گزینه ی `colored` فعال باشد، نماد فعلی زیر هد با رنگ متفاوت نمایش داده می شود تا محل قرارگیری هد واضح تر باشد. اگر این گزینه فعال نباشد، نوار به صورت ساده و بدون رنگ چاپ می شود. این بخش به خوانا تر شدن خروجی کمک می کند، مخصوصاً وقتی ماشین چندین مرحله اجرا می شود و لازم است تغییر موقعیت هد روی نوار دنبال شود.

در ادامه، اگر مقدار `action` برابر `None` باشد، یعنی ماشین هنوز متوقف نشده و در حال اجراست. در این حالت شماره ی مرحله، حالت فعلی، نماد زیر هد و نوار فعلی چاپ می شود. اما اگر `action` مقدار دیگری داشته باشد، یعنی ماشین به حالت پذیرش یا رد رسیده است؛ در نتیجه مقدار `halted` برابر `True` می شود و اجرای متد پایان می یابد. در انتها نیز اگر ماشین در تعداد گام مشخص شده متوقف نشده باشد، با استفاده از `logging.warning` هشدار داده می شود که اجرای `debug` به دلیل رسیدن به محدودیت گام ها متوقف شده است.

بخش اول

ماشین اول:

```
4 transitions = {
5     ('q0', '#'): ('saw_#', '#', 'R'),
6     ('saw_#', '#'): ('saw_##', '#', 'R'),
7     ('saw_##', ''): ('qa', '', 'R'),
8 }
9 # This turing machine accepts only the ## string. It rejects on any other input
10 # computation:
11 # - run ('q0', '#'), it mean the string must start with #, otherwise reject
12 # - if satisfy it move to 'saw_#' state and read next char if it's #, go 'saw_##', otherwise reject
13 # - if satisfy it read next char if it's '' then accept
14 # - ('saw_##', ''), it mean after saw_## it end, so string must be "##"
15
16 def print_states(transition_mapping):
17     states = set()
18
19     for (start, finish) in transition_mapping.items():
20         (s1, _) = start
21         (s2, _, _) = finish
22
23         states.add(s1)
24         states.add(s2)
25
26     print("The Turing machine has", len(states), "states:")
27     for i in states:
28         print(i)
29     print()
30
31 if __name__ == "__main__":
32     print_states(transitions)
33
34
35     machine = TuringMachine(transitions)
36
37     def run(input_):
```

برای این کار، تابع `print_states` نوشته شد. این تابع دیکشنری انتقال‌ها را بررسی می‌کند و از هر `transition`، حالت مبدأ و حالت مقصد را استخراج می‌کند. سپس این حالت‌ها داخل یک `set` قرار داده می‌شوند تا حالت‌های تکراری حذف شوند. در پایان، تعداد حالت‌ها و نام آن‌ها چاپ می‌شود. این بخش برای پاسخ به قسمت «الف» پروژه استفاده شده است که خواسته بود `state`های ماشین تورینگ فهرست شوند.

در بخش اجرای برنامه نیز تابع `print_states(transitions)` فراخوانی شد تا قبل از اجرای تست‌ها، حالت‌های ماشین نمایش داده شوند. همچنین داخل تابع `run` بعد از مشخص شدن نتیجه‌ی پذیرش یا رد ورودی، دستور `machine.debug(w)` اضافه شد. این دستور باعث می‌شود پیکربندی ماشین در هر مرحله از اجرا چاپ شود و بتوان وضعیت نوار، حالت فعلی و نماد زیر هد را در طول

transitionها مشاهده کرد. این بخش مربوط به قسمت «ب» پروژه است که خواسته بود configuration ماشین به ازای هر انتقال قابل بررسی باشد.

در انتهای فایل، چند ورودی مختلف اجرا شده‌اند تا رفتار ماشین بررسی شود. با توجه به قوانین این ماشین واضح است که تنها ورودی ## پذیرفته می‌شود و ورودی‌های دیگر رد می‌شوند.

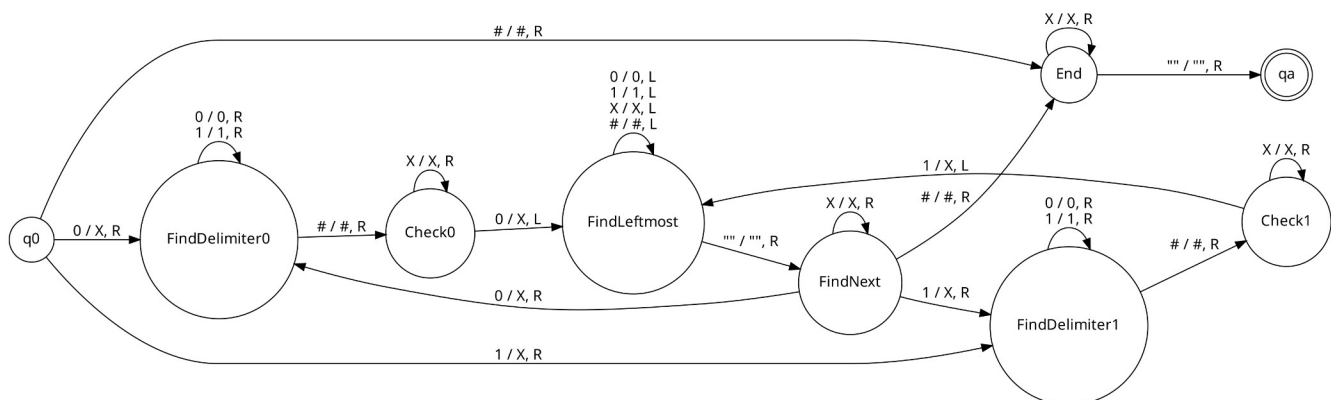
ماشین دوم:

```
4 transitions = {
5     ('q0', '#'): ('saw_#', '#', 'R'),
6     ('saw_#', '#'): ('saw_##', '#', 'R'),
7     ('saw_##', ''): ('qa', '', 'R'),
8 }
9 # This turing machine accepts only the ## string. It rejects on any other input
10 # computation:
11 # - run ('q0', '#'), it mean the string must start with #, otherwise reject
12 # - if satisfy it move to 'saw_#' state and read next char if it's #, go 'saw_##', otherwise reject
13 # - if satisfy it read next char if it's '' then accept
14 # - ('saw_##', ''), it mean after saw_## it end, so string must be "##"
15
16 def print_states(transition_mapping):
17     states = set()
18
19     for (start, finish) in transition_mapping.items():
20         (s1, _) = start
21         (s2, _, _) = finish
22
23         states.add(s1)
24         states.add(s2)
25
26     print("The Turing machine has", len(states), "states:")
27     for i in states:
28         print(i)
29     print()
30
31 if __name__ == "__main__":
32     print_states(transitions)
33
34
35     machine = TuringMachine(transitions)
36
37     def run(input_):
```

در ماشین دوم نیز برای پاسخ به بخش مربوط به فهرست کردن حالت‌ها، از همان تابع `print_states` که در فایل ماشین اول نوشته شده بود استفاده شده است. این تابع در ابتدای فایل `import` شده و قبل از اجرای تست‌ها روی دیکشنری `transitions` فراخوانی می‌شود. با این کار، تمام حالت‌های استفاده‌شده در ماشین دوم به صورت خودکار استخراج و چاپ می‌شوند و نیازی نیست `state` ها به صورت دستی نوشته شوند.

همچنین برای مشاهده‌ی پیکربندی ماشین در هر مرحله، داخل تابع `run` از دستور `machine.debug(w,colored=True)` استفاده شده است. این دستور باعث می‌شود اجرای ماشین به صورت مرحله‌به‌مرحله نمایش داده شود و بتوان در هر `transition` وضعیت فعلی ماشین، نماد زیر هد و محتوای نوار را مشاهده کرد. فعال بودن گزینه‌ی `colored=True` نیز کمک می‌کند محل قرارگیری هد روی نوار واضح‌تر دیده شود.

توصیف:



این ماشین تورینگ زبان $W\#W$ را توصیف می‌کند؛ یعنی رشته‌هایی را می‌پذیرد که در دو طرف علامت $\#$ دقیقاً یک رشته‌ی یکسان قرار گرفته باشد. به بیان دقیق‌تر، زبان ماشین به صورت زیر است:

$$L = \{w\#w \mid w \in \{0,1\}^*\}$$

در این زبان، w می‌تواند هر رشته‌ای از نمادهای 0 و 1 باشد. بنابراین اگر قبل و بعد از $\#$ دقیقاً یک رشته‌ی برابر داشته باشیم، ورودی پذیرفته می‌شود. برای مثال رشته‌های $\#$ ، $0\#0$ ، $1\#1$ ، $01\#01$ و $101\#101$ پذیرفته می‌شوند. اما رشته‌هایی مثل $1\#0$ ، $1\#11$ یا $10\#01$ رد می‌شوند، چون دو طرف $\#$ با هم برابر نیستند.

ماشین از حالت q_0 شروع می‌کند. در این حالت، اولین نماد بررسی‌نشده از سمت چپ $\#$ خوانده می‌شود. اگر این نماد 0 باشد، ماشین آن را با X علامت‌گذاری می‌کند و وارد حالت `FindDelimiter0` می‌شود. اگر این نماد 1 باشد، آن را با X علامت‌گذاری می‌کند و وارد حالت `FindDelimiter1` می‌شود. بنابراین حالت q_0 وظیفه دارد نماد بعدی از رشته‌ی سمت چپ را انتخاب کند.

در حالت‌های FindDelimiter^۰ و FindDelimiter^۱، ماشین به سمت راست حرکت می‌کند تا به علامت جداکننده‌ی # برسد. نام این حالت‌ها نشان می‌دهد که ماشین در حال پیدا کردن delimiter است و همزمان در حافظه‌ی حالت خود نگه می‌دارد که نماد انتخاب‌شده از سمت چپ ۰ بوده یا ۱.

بعد از رسیدن به #، ماشین وارد یکی از حالت‌های Check^۰ یا Check^۱ می‌شود. اگر نماد انتخاب‌شده از سمت چپ ۰ بوده باشد، وارد Check^۰ می‌شود و در سمت راست # باید یک ۰ متناظر پیدا کند. اگر نماد انتخاب‌شده ۱ بوده باشد، وارد Check^۱ می‌شود و باید یک ۱ متناظر پیدا کند. وقتی نماد متناظر پیدا شد، آن هم به X تبدیل می‌شود تا مشخص شود این نماد قبلاً بررسی شده است.

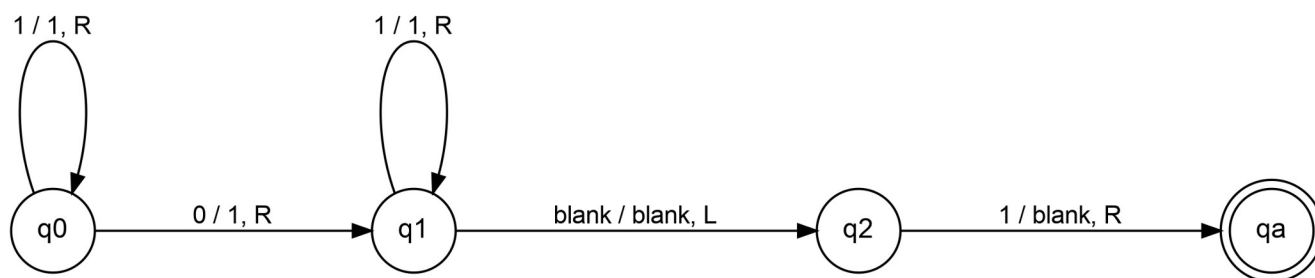
بعد از علامت‌گذاری نماد متناظر در سمت راست، ماشین وارد حالت FindLeftmost می‌شود. در این حالت، ماشین به سمت چپ حرکت می‌کند تا به ابتدای نوار برسد. سپس در حالت FindNext دوباره از چپ به راست حرکت می‌کند و از روی Xهای قبلی عبور می‌کند تا اولین نماد بررسی‌نشده‌ی بعدی در سمت چپ را پیدا کند. اگر نماد بعدی ۰ باشد، دوباره وارد مسیر FindDelimiter^۰ می‌شود و اگر ۱ باشد وارد مسیر FindDelimiter^۱ می‌شود.

این روند ادامه پیدا می‌کند تا زمانی که در حالت FindNext ماشین به # برسد. رسیدن به # در این مرحله یعنی همه‌ی نمادهای سمت چپ بررسی و با X علامت‌گذاری شده‌اند. سپس ماشین وارد حالت End می‌شود و سمت راست # را بررسی می‌کند. اگر در سمت راست فقط Xهای علامت‌گذاری‌شده باقی مانده باشند و سپس نماد خالی دیده شود، ماشین وارد حالت پذیرش qa می‌شود.

در نتیجه، این ماشین با استفاده از نماد کمکی X، هر بار یک نماد از سمت چپ را با نماد متناظر آن در سمت راست مقایسه می‌کند. اگر تمام نمادها از نظر مقدار و ترتیب برابر باشند، رشته پذیرفته می‌شود. اگر در هر مرحله نماد متناظر پیدا نشود یا مقدار آن متفاوت باشد، ماشین انتقال مناسبی ندارد و ورودی رد می‌شود. بنابراین زبان پذیرفته‌شده توسط این ماشین دقیقاً W[#]W است.

بخش دوم

ماشین Adder :



ماشین از حالت q_0 شروع می‌کند. در این حالت، تا زمانی که نماد زیر هد ۱ باشد، ماشین همان ۱ را نگه می‌دارد و به سمت راست حرکت می‌کند. این کار باعث می‌شود ماشین از روی عدد اول عبور کند تا به جداکننده‌ی ۰ برسد.

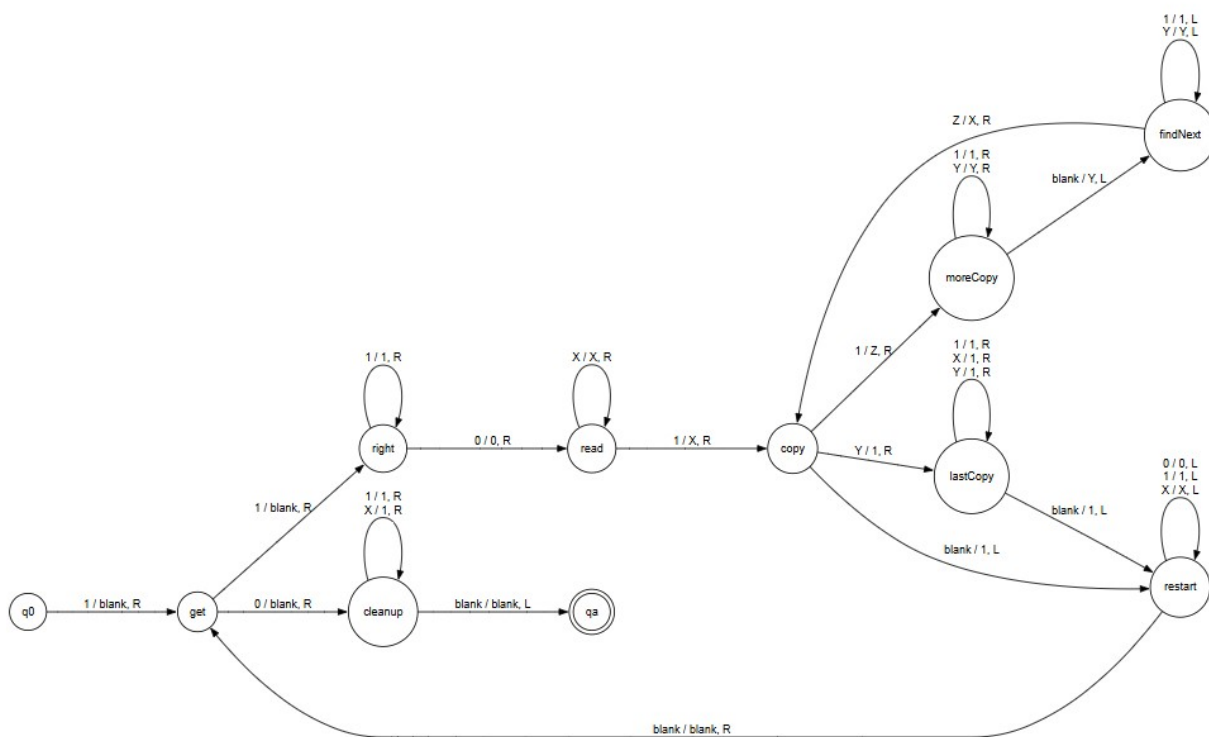
وقتی ماشین در حالت q_0 به نماد ۰ می‌رسد، آن را به ۱ تبدیل می‌کند و وارد حالت q_1 می‌شود. این مرحله در واقع دو بخش عدد را به هم وصل می‌کند، چون جداکننده‌ی بین دو عدد حذف شده و به جای آن یک ۱ قرار می‌گیرد.

در حالت q_1 ، ماشین از روی همه‌ی ۱‌های عدد دوم عبور می‌کند و به سمت راست می‌رود تا به نماد خالی انتهای نوار برسد. وقتی به نماد خالی رسید، وارد حالت q_2 می‌شود و یک خانه به سمت چپ برمی‌گردد.

در حالت q_2 ، ماشین آخرین ۱ نوار را پاک می‌کند و سپس وارد حالت پذیرش q_a می‌شود. دلیل این کار این است که در مرحله‌ی قبل، جداکننده‌ی ۰ به ۱ تبدیل شده بود و این باعث می‌شد تعداد ۱‌ها یکی بیشتر از مجموع واقعی شود. با حذف آخرین ۱، تعداد نهایی ۱‌ها دقیقاً برابر با حاصل جمع دو عدد می‌شود.

برای مثال، در ورودی ۱۱۰۱۱۱ ابتدا ۰ وسط به ۱ تبدیل می‌شود و نوار به شکل ۱۱۱۱۱۱ درمی‌آید. سپس ماشین آخرین ۱ را حذف می‌کند و خروجی نهایی ۱۱۱۱ می‌شود که همان نتیجه‌ی $3+2$ است. همچنین برای ورودی ۱۱۱۰۱۱۱۱، خروجی نهایی ۱۱۱۱۱۱ به دست می‌آید که نشان‌دهنده‌ی $4+3$ است.

ماشین Multiplier :



در این بخش، ماشین تورینگ ضرب یکانی پیاده‌سازی شده است. ورودی ماشین شامل دو عدد یکانی است که با نماد ۰ از هم جدا شده‌اند. برای مثال، ورودی ۱۱۰۱۱۱ نشان‌دهنده‌ی 3×2 است؛ یعنی دو ۱ قبل از ۰ و سه ۱ بعد از آن قرار دارد. هدف ماشین این است که در پایان، حاصل ضرب دو عدد را به صورت تعدادی ۱ روی نوار باقی بگذارد.

ایده‌ی اصلی این ماشین این است که به ازای هر ۱ در عدد اول، یک نسخه از عدد دوم در انتهای نوار ساخته شود. بنابراین اگر عدد اول ۲ و عدد دوم ۳ باشد، ماشین دو بار عدد دوم را کپی می‌کند و در نهایت ۶ تا ۱ تولید می‌شود.

ماشین از حالت q_0 شروع می‌کند. در این حالت، اولین ۱ از عدد اول پاک می‌شود و ماشین وارد حالت `get` می‌شود. حالت `get` بررسی می‌کند که آیا هنوز از عدد اول ۱ باقی مانده یا نه. اگر هنوز ۱ باقی مانده باشد، یکی از آن‌ها را پاک می‌کند و وارد حالت `right` می‌شود تا به سمت بخش عدد دوم حرکت کند. اگر به نماد جداکننده‌ی ۰ برسد، یعنی همه‌ی ۱های عدد اول پردازش شده‌اند و ماشین وارد مرحله‌ی `cleanup` می‌شود.

در حالت `right`، ماشین از روی باقی‌مانده‌ی عدد اول عبور می‌کند تا به نماد ۰ برسد. بعد از رسیدن به ۰، وارد حالت `read` می‌شود. حالت `read` در سمت راست جداکننده حرکت می‌کند و به دنبال ۱های عدد دوم می‌گردد. هر ۱ از عدد دوم که قرار است کپی شود، ابتدا به X تبدیل می‌شود تا مشخص شود آن نماد در این دور بررسی شده است.

بعد از علامت‌گذاری یک ۱ در حالت `read`، ماشین وارد حالت `copy` می‌شود. در این بخش، ماشین تلاش می‌کند یک ۱ متناظر در انتهای نوار اضافه کند. اگر در مسیر به ۱ دیگری برسد، آن را موقتاً با Z علامت‌گذاری می‌کند و وارد حالت `moreCopy` می‌شود تا به انتهای نوار برسد. در انتهای نوار، یک Y نوشته می‌شود. نمادهای Y در واقع خروجی‌های ساخته‌شده‌ی حاصل ضرب هستند که در مراحل بعدی مدیریت می‌شوند.

حالت `findNext` ماشین را از انتهای نوار به سمت چپ برمی‌گرداند تا به نماد Z برسد. وقتی Z پیدا شد، دوباره به X تبدیل می‌شود و ماشین وارد حالت `copy` می‌شود تا ادامه‌ی کپی‌کردن انجام شود. با این روند، ماشین می‌تواند تمام ۱های عدد دوم را برای یک مرحله از پردازش عدد اول کپی کند.

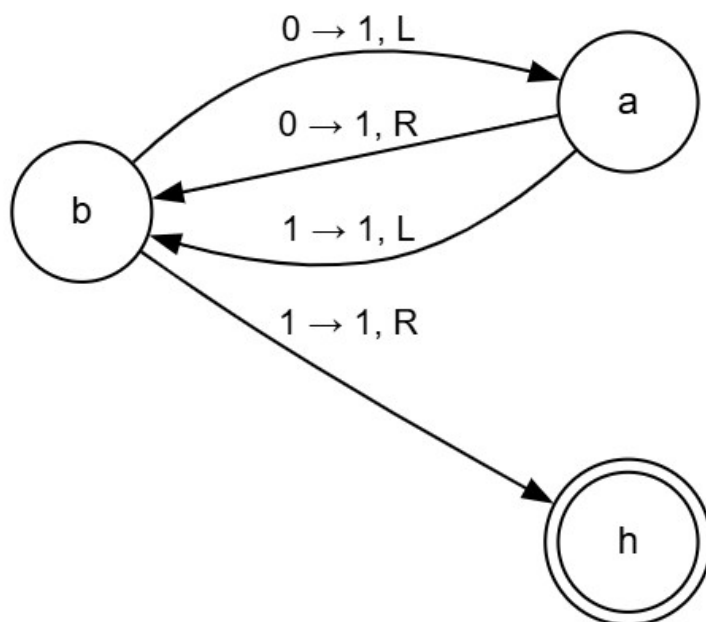
وقتی ماشین به پایان کپی‌کردن یک دور برسد، وارد حالت `restart` می‌شود. در حالت `restart`، ماشین به سمت چپ حرکت می‌کند تا دوباره به ابتدای نوار برسد. سپس وارد حالت `get` می‌شود تا بررسی کند آیا هنوز از عدد اول ۱ پردازش‌نشده‌ای باقی مانده است یا نه. اگر باقی مانده باشد، یک دور دیگر کپی عدد دوم انجام می‌شود.

وقتی همه‌ی ۱های عدد اول پردازش شدند، ماشین در حالت `get` به جداکننده‌ی ۰ می‌رسد و وارد حالت `cleanup` می‌شود. در این مرحله، نمادهای کمکی باقی‌مانده مثل X به ۱ تبدیل می‌شوند و بخش‌های اضافی پاک‌شده یا خالی‌شده دیگر در خروجی نهایی دیده نمی‌شوند. سپس ماشین به حالت پذیرش `qa` می‌رود.

در نتیجه، این ماشین ضرب را با تکرار عمل کپی‌کردن انجام می‌دهد. عدد اول مشخص می‌کند که عدد دوم چند بار باید کپی شود، و خروجی نهایی تعداد ۱هایی است که برابر با حاصل ضرب دو عدد ورودی است. برای مثال، ورودی ۱۱۰۱۱۱ که نشان‌دهنده‌ی 2×3 است، در پایان به ۱۱۱۱۱۱ تبدیل می‌شود. همچنین ورودی ۱۱۱۰۱۱۱۱ که نشان‌دهنده‌ی 3×4 است، خروجی ۱۱۱۱۱۱۱۱۱۱۱ تولید می‌کند.

بخش سوم

بیدستر دو حالت:



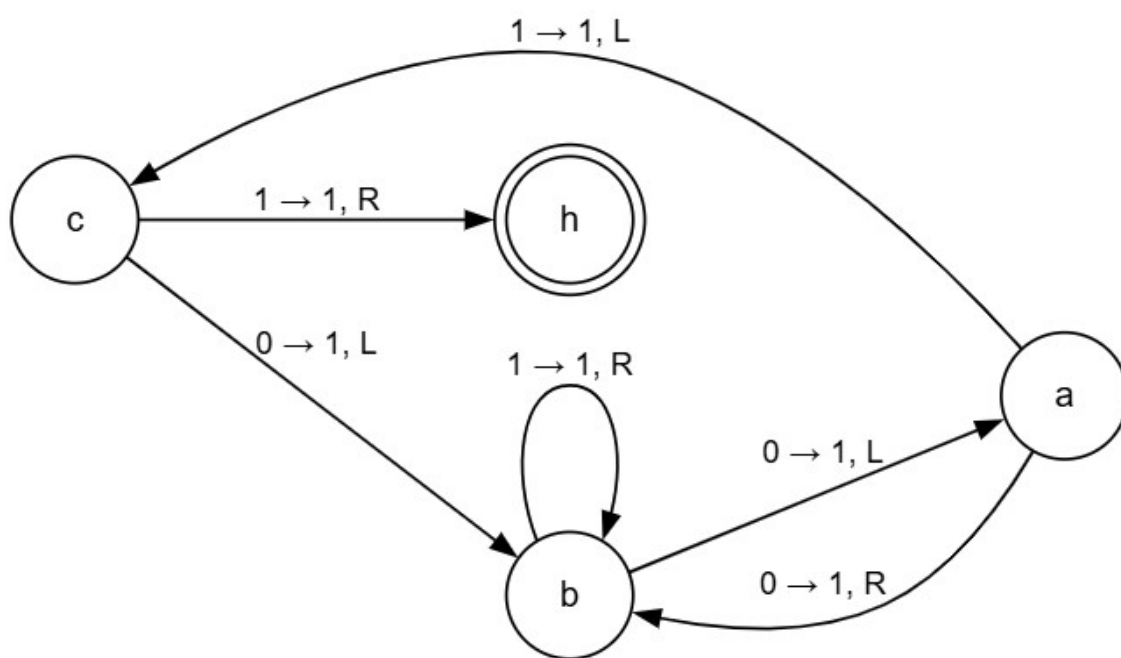
توضیح:

این ماشین دارای دو حالت کاری a و b است و حالت h برای توقف استفاده می‌شود. این ماشین در پایان ۴ تا ۱ روی نوار تولید می‌کند و بعد از ۶ حرکت متوقف می‌شود. ساختار گراف آن خیلی ساده است، چون تقریباً تمام رفتار ماشین بین دو حالت a و b اتفاق می‌افتد. این دو حالت مثل یک چرخه‌ی رفت و برگشتی عمل می‌کنند؛ یعنی ماشین با حرکت بین آن‌ها، خانه‌های خالی اطراف نقطه‌ی شروع را به ۱ تبدیل می‌کند و بعد دوباره روی خانه‌هایی که قبلاً تغییر داده برمی‌گردد.

در این ماشین، حالت a بیشتر نقش شروع کننده‌ی حرکت را دارد و حالت b نقش کامل کننده‌ی همان حرکت را بازی می‌کند. وقتی ماشین روی خانه‌های خالی قرار می‌گیرد، آن‌ها را به ۱ تبدیل می‌کند و جهت حرکت را عوض می‌کند. به همین دلیل، به جای اینکه فقط در یک جهت جلو برود، یک حرکت رفت و برگشتی کوتاه روی نوار دارد. در گراف، یال‌هایی که بین a و b هستند همین رفت و برگشت را نشان می‌دهند. مسیر توقف هم از حالت b به h است؛ یعنی ماشین وقتی در حالت b به یک وضعیت مشخص روی نوار برسد، دیگر وارد چرخه‌ی قبلی نمی‌شود و به حالت توقف می‌رود. بنابراین شهود اصلی این ماشین این است که با یک چرخه‌ی خیلی کوچک بین دو حالت، چند خانه را ۱ می‌کند و بعد از همان چرخه خارج می‌شود

این ماشین، بهینه‌ترین ماشین ممکن برای ۲ استییت است

بیدستر سه‌حالت:



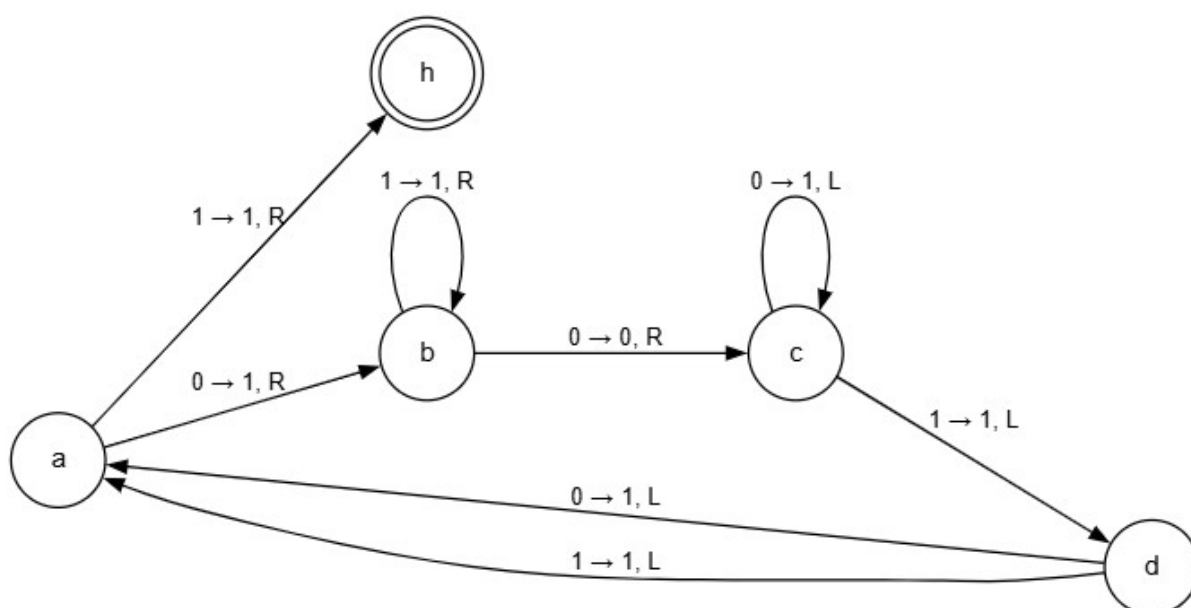
توضیح:

این ماشین دارای سه حالت کاری a ، b و c است و در حالت h متوقف می‌شود. این ماشین در پایان ۶ تا ۱ تولید می‌کند و بعد از ۱۳ حرکت به توقف می‌رسد. اضافه شدن حالت سوم باعث می‌شود رفتار ماشین نسبت به بیدستر دو حالت کمتر مستقیم و کمی هوشمندانه‌تر شود. در اینجا ماشین فقط بین دو حالت رفت و برگشت نمی‌کند، بلکه یک حالت جداگانه برای کنترل مسیر برگشت و توقف دارد. در گراف این ماشین، حالت a نقطه‌ی اصلی تصمیم‌گیری است. از a ماشین می‌تواند وارد مسیر b شود یا به سمت c برود. حالت b بیشتر نقش حرکت روی بخش ساخته‌شده‌ی نوار را دارد. چون روی مقدار ۱ یک یال از b به خودش وجود دارد، ماشین می‌تواند روی یک رشته از ۱ها به حرکت ادامه دهد، بدون اینکه سریع متوقف شود. این self-loop در گراف نشان می‌دهد که ماشین ممکن است چند خانه پشت سر هم را رد کند و همچنان در همان منطق حرکتی باقی بماند.

حالت c نقش مهم‌تری در پایان دادن به کار دارد. این حالت مثل بخشی از ماشین است که بعد از برگشتن روی نوار، بررسی می‌کند آیا ماشین باید دوباره وارد چرخه‌ی تولید ۱ شود یا متوقف شود. در گراف، یال $h \rightarrow c$ نشان‌دهنده‌ی مسیر توقف است. یعنی توقف مستقیماً از حالت شروع یا حالت حرکت اصلی اتفاق نمی‌افتد، بلکه بعد از اینکه ماشین وارد حالت کنترل‌کننده‌ی c شد و وضعیت مناسب را دید، به h می‌رود. به همین خاطر، شهود گراف این است که ماشین ابتدا با a و b نوار را گسترش می‌دهد، سپس با c مسیر برگشت و توقف را کنترل می‌کند.

این ماشین، بهینه‌ترین ماشین ممکن برای ۳ استیت است

بیدستر چهارحالت:



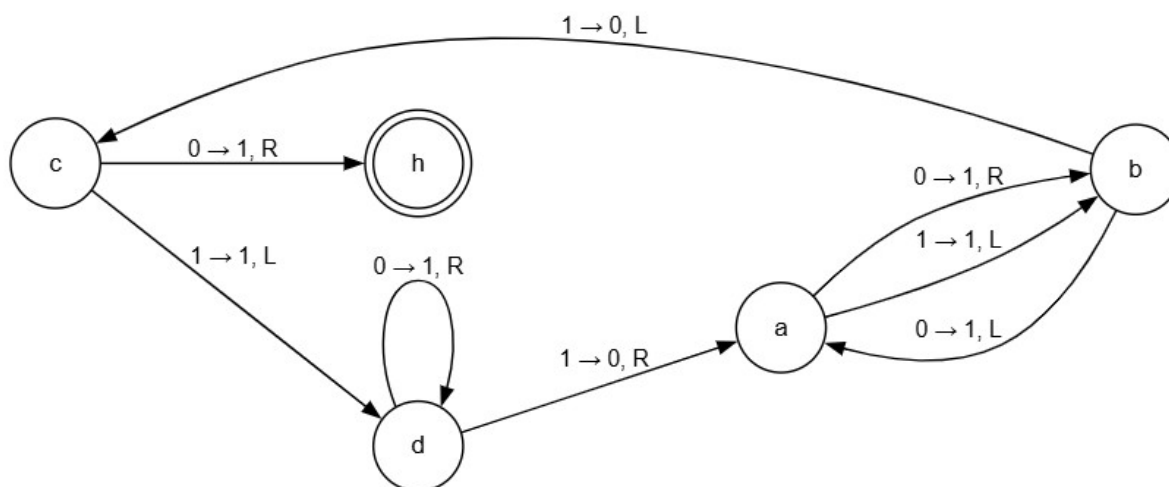
توضیح:

این ماشین دارای چهار حالت کاری a, b, c و d است و در حالت h متوقف می‌شود. این ماشین در پایان ۷ تا ۱ تولید می‌کند و بعد از ۱۷ حرکت متوقف می‌شود. منطق گراف این ماشین بر اساس یک چرخه‌ی نسبتاً مشخص ساخته شده است. اگر به گراف نگاه کنیم، مسیر اصلی ماشین به شکل یک چرخش بین حالت‌های a, b, c و d دیده می‌شود. یعنی ماشین چند بار وارد این مسیر می‌شود، نوار را تغییر می‌دهد و دوباره به ابتدای چرخه برمی‌گردد.

حالت a در این ماشین دو نقش دارد: هم حالت شروع است و هم مسیر توقف از آن جدا می‌شود. این موضوع در گراف با $a \rightarrow h$ مشخص است. یعنی ماشین وقتی دوباره به a برمی‌گردد، بسته به نمادی که زیر هد قرار دارد، یا چرخه را ادامه می‌دهد یا متوقف می‌شود. حالت b بیشتر حالت حرکت به سمت راست است. وجود self-loop روی b نشان می‌دهد که ماشین اگر روی ۱ها قرار بگیرد، می‌تواند در همان حالت به حرکت ادامه دهد و از بخش نوشته‌شده‌ی نوار عبور کند.

حالت c جایی است که ماشین دوباره شروع به تغییر دادن نوار می‌کند. این حالت بعد از عبور از بخش قبلی، وارد عمل می‌شود و باعث می‌شود مقادیر جدیدی روی نوار ساخته شوند. حالت d هم نقش برگشت‌دهنده دارد؛ یعنی بعد از اینکه ماشین در c وضعیت لازم را ایجاد کرد، d آن را به سمت a برمی‌گرداند. از دید گراف، مسیر اصلی ماشین را می‌شود به صورت چرخه‌ی $a \rightarrow b \rightarrow c \rightarrow d \rightarrow a$ دید. توقف زمانی اتفاق می‌افتد که این چرخه به a برگردد، اما این بار شرایط نوار طوری باشد که به جای ادامه دادن چرخه، ماشین از a به h خارج شود. بنابراین این ماشین نسبت به قبلی‌ها ساختار چرخه‌ای واضح‌تری دارد و همین چرخه باعث تولید ۷ تا ۱ قبل از توقف می‌شود.

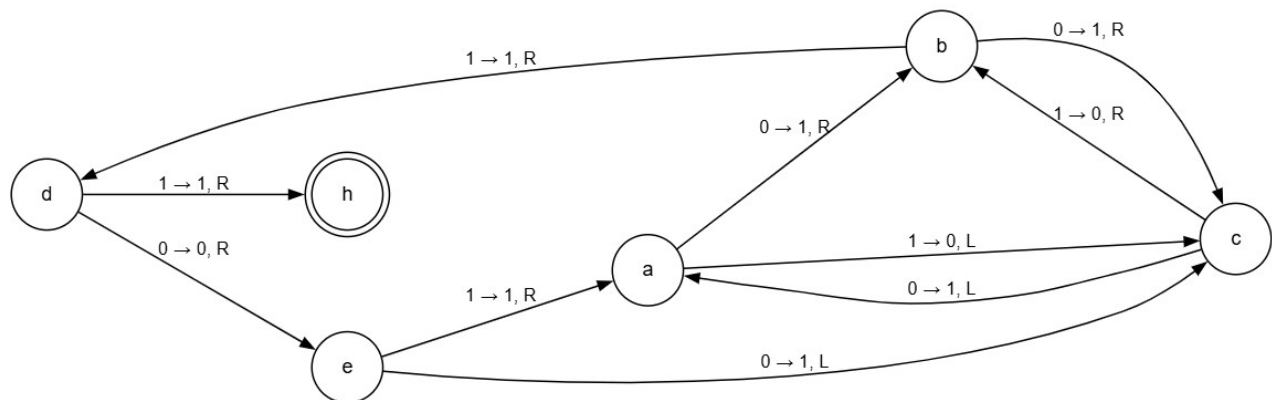
بهینه‌ترین حالت ممکن:



۱۰۷ استپ

۱۳ عدد یک

بیدستر پنج حالت:



توضیح:

این ماشین دارای پنج حالت کاری a, b, c, d و e است و در حالت h متوقف می‌شود. این ماشین در پایان ۱ تا ۵۰۱ تولید می‌کند و بعد از ۱۳۴۴۶۷ حرکت متوقف می‌شود. دلیل زیاد شدن تعداد حرکات این است که گراف این ماشین فقط یک چرخه‌ی ساده ندارد، بلکه چند چرخه‌ی داخلی دارد که ماشین را بارها به بخش‌های قبلی برمی‌گردانند. به همین خاطر، ماشین قبل از رسیدن به حالت توقف، مدت زیادی روی نوار رفت و برگشت انجام می‌دهد.

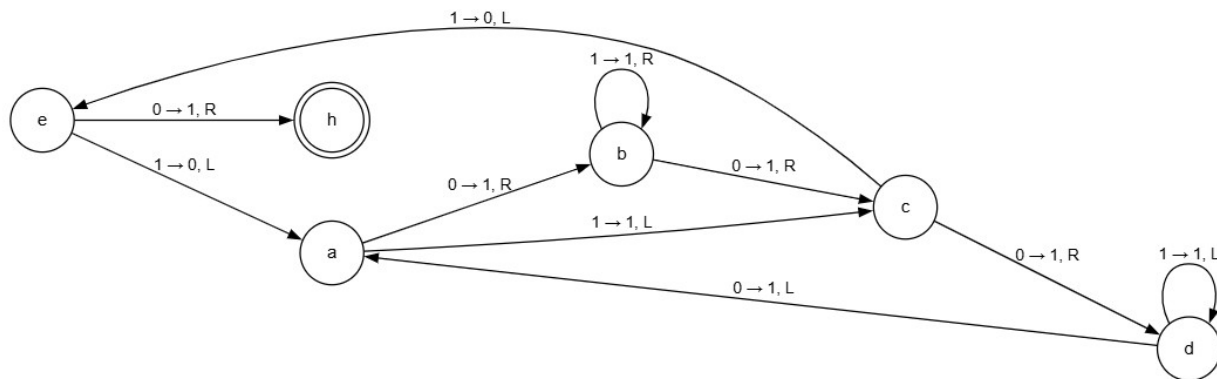
در این ماشین، حالت‌های a, b و c بخش اصلی چرخه‌های تولید و تغییر نوار را می‌سازند. حالت a فقط شروع‌کننده نیست؛ بلکه یکی از نقاط تصمیم‌گیری مهم ماشین است. از a ماشین می‌تواند وارد مسیر b شود یا در شرایط دیگر وارد c شود. نکته‌ی مهم این است که این ماشین همیشه فقط ۰ را به ۱ تبدیل نمی‌کند. در بعضی یال‌ها، مقدار ۱ به ۰ تبدیل می‌شود. همین بازنویسی باعث می‌شود ماشین یک الگوی ساده و خطی نسازد، بلکه مدام بخشی از نوار را تغییر دهد و دوباره وارد چرخه‌های قبلی شود.

حالت b در گراف مثل یک حالت انتقالی مهم عمل می‌کند. از b ماشین یا به c می‌رود، یا وارد d می‌شود. وقتی ماشین وارد d می‌شود، به مسیر نزدیک‌تر به توقف رسیده است، چون تنها یال توقف ماشین از d به h است. اما d همیشه توقف نمی‌کند؛ اگر شرط توقف برقرار نباشد، ماشین از d به e می‌رود و دوباره وارد چرخه‌های دیگر می‌شود. بنابراین d مثل یک حالت بررسی‌کننده است: یا ماشین را متوقف می‌کند، یا آن را به ادامه‌ی پردازش می‌فرستد.

حالت e هم باعث طولانی‌تر شدن رفتار ماشین می‌شود. این حالت ماشین را دوباره به c یا a برمی‌گرداند و اجازه نمی‌دهد ماشین سریع به توقف برسد. به زبان گرافی، مسیریابی مثل $c \rightarrow e \rightarrow d \rightarrow a$ و $e \rightarrow a$ باعث می‌شوند ماشین دوباره وارد بخش‌های قبلی گراف شود. همین برگشت‌ها دلیل اصلی تعداد زیاد $step$ ها هستند. در واقع گراف این ماشین چند حلقه‌ی تو در تو دارد: یک حلقه بین a, b, c و یک مسیر دیگر که از b به d و e می‌رود و دوباره ماشین را به چرخه برمی‌گرداند. توقف فقط زمانی اتفاق می‌افتد که ماشین در حالت d روی

نمادی قرار بگیرد که یال $d \rightarrow h$ فعال شود. به همین دلیل، این ماشین قبل از توقف تعداد زیادی بازنویسی و جابه‌جایی انجام می‌دهد و در نهایت ۵۰۱ تا ۱ باقی می‌گذارد.

بهینه‌ترین حالت شناخته‌شده:



۴۷۱۷۶۸۷ استپ

۴۰۹۸ عدد یک

چرا پیدا کردن برنامه‌های بیدستر پرتلاش کار ساده‌ای نیست و چرا این مسئله به عنوان یک مسئله غیرقابل حل (**undecidable**) شناخته می‌شود؟

دو نوع Busy Beaver داریم که معمولاً کنار هم مطرح می‌شوند:

یکی می‌گویند بیشترین تعداد ۱ که یک ماشین n حالت می‌تواند قبل از توقف بنویسد چقدر است. به این معمولاً می‌گویند $\Sigma(n)$.

یکی می‌گویند بیشترین تعداد گام‌هایی که یک ماشین n حالت می‌تواند قبل از توقف اجرا شود چقدر است. به این معمولاً می‌گویند $S(n)$.

آن چیزی که مستقیم به ما کران بالا برای زمان اجرا می‌دهد، نوع دوم است، یعنی:

$S(n)$ = بیشترین تعداد گام بین تمامی ماشین‌های n حالتی که بالاخره متوقف می‌شوند

چرا این کران بالاست؟

چون تعریف $S(n)$ این است که بین همه‌ی ماشین‌های n حالت‌ای که $halt$ می‌کنند، طولانی‌ترین اجرای ممکن را پیدا می‌کند. یعنی هیچ ماشین n حالت‌ی متوقف‌شونده‌ای نمی‌تواند بیشتر از $S(n)$ گام اجرا شود.

مثلاً اگر برای ماشین‌های ۴ حالت داشته باشیم:

$$S(4) = 107$$

معنی‌اش این است:

هر ماشین ۴ حالت‌ای که قرار است متوقف شود، حتماً در حداکثر ۱۰۷ گام متوقف می‌شود.

پس اگر یک ماشین ۴ حالت را ۱۰۷ گام اجرا کنیم و هنوز $halt$ نکرده باشد، دیگر نمی‌تواند جزو ماشین‌های متوقف‌شونده باشد. چون اگر متوقف‌شونده بود، طبق تعریف $S(4)$ باید تا ۱۰۷ گام متوقف می‌شد. پس نتیجه می‌گیریم که آن ماشین هیچ‌وقت $halt$ نمی‌کند.

اما اگر منظورمان از Busy Beaver فقط تعداد ۱ها باشد، یعنی $\Sigma(n)$ ، آن به طور مستقیم کران زمانی نمی‌دهد. چون ممکن است ماشینی تعداد کمی ۱ بنویسد ولی خیلی زیاد حرکت کند. برای بحث حل مسئله‌ی توقف، معمولاً از نسخه‌ی زمانی Busy Beaver یعنی $S(n)$ استفاده می‌کنند.

بنابراین:

اگر Busy Beaver را به معنی بیشترین تعداد گام تا توقف بگیریم، عدد آن مستقیماً کران بالا برای اجرای ماشین‌های متوقف‌شونده است.

چون اگر ماشینی با n حالت قرار باشد $halt$ کند، نمی‌تواند بیشتر از $S(n)$ گام طول بکشد.

بنابراین اگر بعد از $S(n)$ گام هنوز $halt$ نکرد، یعنی هرگز $halt$ نمی‌کند.

به همین دلیل اگر $S(n)$ را برای هر n بلد بودیم، می‌توانستیم مسئله‌ی توقف را حل کنیم.

چون مسئله‌ی توقف غیرقابل حل است، محاسبه‌ی کلی Busy Beaver هم غیرقابل حل است.